

ЛАБОРАТОРНАЯ РАБОТА №4

Вычисление наибольшего общего делителя

Хамза хуссен

Содержание

1	Цель работы	3
2	Задание	4
3	Теоретическое введение(Алгоритмы)	5
3.1	алгоритм Евклида	5
3.2	бинарный алгоритм Евклида	5
3.3	расширенный алгоритм Евклида.	6
3.4	Расширенный бинарный алгоритм Евклида.	6
4	Выполнение лабораторной работы	8
4.1	алгоритм Евклида	8
4.2	бинарный алгоритм Евклида	9
4.3	расширенный алгоритм Евклида	10
4.4	расширенный бинарный алгоритм Евклида.	11
5	Выводы	15
	Список литературы	16

1 Цель работы

Изучение и практическое применение методов программной реализации Алгоритмов вычисления наибольшего общего делителя.

2 Задание

1. Реализовать алгоритм Евклида.
2. Реализовать бинарный алгоритм Евклида.
3. Реализовать расширенный алгоритм Евклида.
4. Реализовать расширенный бинарный алгоритм Евклида.

3 Теоретическое введение(Алгоритмы)

3.1 алгоритм Евклида

Вход. Целые числа a, b ; $0 < b \leq a$.

Выход. $d = \text{НОД}(a, b)$.

1. Положить $r_0 = a, r_1 = b, i = 1$.
2. Найти остаток r_{i+1} от деления r_{i-1} на r_i ; Чернышевского
3. Если $r_{i+1} = 0$, то положить $d = r_i$. В противном случае положить $i = i + 1$ и вернуться на шаг 2.
4. Результат: d .

3.2 бинарный алгоритм Евклида

Вход. Целые числа a, b ; $0 < b \leq a$.

Выход. $d = \text{НОД}(a, b)$.

1. Положить $d = 1$.
2. Пока оба числа a и b четные, выполнять $a \leftarrow a/2, b \leftarrow b/2$ до получения хотя бы одного нечетного значения a или b .

3. Положить $u < a, v < b$.
4. Пока $u \neq 0$ выполнять следующие действия:
 - 4.1. Пока u нечетное, полагать $u <- u / 2$
 - 4.2. Пока v четное, полагать $v <- v / 2$
 - 4.3. Если $u \geq v$ положить $u <- u - v$. В противном случае положить $v <- v - u$.
5. Положить $d <- gv$
6. Результат: d

3.3 расширенный алгоритм Евклида.

Вход. Целые числа a, b ; $0 < b \leq a$.

Выход. $d = \text{НОД}(a, b)$.

1. положить $r_0 <- a, r_1 <- b, x_0 <- 1, x_1 <- 0, y_0 <- 0, y_1 <- 1, i <- 1$
2. Разделить с остатком r_{i-1} на r_i : $r_{i-1} = q_i r_i + r_{i+1}$.
3. Если $r_{i+1} = 0$, то положить $d <- r_i, x <- x_i, y <- y_i$. В противном случае положить $x_{i+1} <- x_{i-1} - q_i x_i, y_{i+1} <- y_{i-1} - q_i y_i, i <- i + 1$ и вернуться на шаг 2.
4. Результат: d, x, y .

3.4 Расширенный бинарный алгоритм Евклида.

Вход. Целые числа a, b ; $0 < b \leq a$.

Выход. $d = \text{НОД}(a, b)$.

1. Положить $g <- 1$.

2. Пока числа a и b четные, выполнять $a \leftarrow 5a - 3b - 29$ до получения хотя бы одного нечетного значения a и b .
3. Положить $i \leftarrow a, v \leftarrow b, A \leftarrow 1, B \leftarrow 0, C \leftarrow 0, D \leftarrow 1$.
4. Пока $i \neq 0$ выполнять следующие действия:
 - 4.1. Пока i четное:
 - 4.1.1. Положить $i \leftarrow i / 2$
 - 4.1.2. Если оба числа A и B четные, то положить $A \leftarrow A / 2, B \leftarrow B / 2$. В противном случае положить $A \leftarrow A + 1$.
 - 4.2. Пока o четное:
 - 4.2.1. Положить $o \leftarrow o - 1$
 - 4.2.2. Если оба числа C и D четные, то положить $C \leftarrow C / 2, D \leftarrow D / 2$. В противном случае положить $C \leftarrow C + 1$.
 - 4.3. При $i \geq v$ положить $i \leftarrow i - v, A \leftarrow A - C, B \leftarrow B - D$. В противном случае положить $0 \leftarrow v - i, C \leftarrow C - A, D \leftarrow D - B$.
5. Положить $d \leftarrow gv, x \leftarrow C, y \leftarrow D$.
6. Результат: d, x, y .

4 Выполнение лабораторной работы

4.1 алгоритм Евклида

```
# Euclid's Algorithm (remainder / division method)
def gcd_euclid(a: int, b: int) -> int:
    """Classic Euclid GCD using remainders."""
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    while b != 0:
        a, b = b, a % b
    return a

gcd = gcd_euclid(45, 10)
print("GCD", gcd)
```

GCD 5

4.2 бинарный алгоритм Евклида

```
#Binary GCD (Stein's algorithm)
def gcd_binary(a: int, b: int) -> int:

    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    # count common factors of 2
    shift = 0
    while ((a | b) & 1) == 0: # both even
        a >>= 1
        b >>= 1
        shift += 1

    # make a odd
    while (a & 1) == 0:
        a >>= 1

    while b != 0:
        # make b odd
        while (b & 1) == 0:
            b >>= 1

    # now a and b are odd
```

```

    if a > b:
        a, b = b, a
    b = b - a # b becomes even (difference of odds)

    return a << shift

gcd = gcd_binary(45, 10)
print(gcd)

```

5

4.3 расширенный алгоритм Евклида

```

#Extended Euclid (returns gcd, x, y)
def extended_gcd_euclid(a: int, b: int):
    """
    Returns (g, x, y) such that:
        g = gcd(a, b)
        a*x + b*y = g
    """
    if b == 0:
        g = abs(a)
        # make x carry the sign of a so identity holds
        x = 1 if a > 0 else -1 if a < 0 else 0
        return g, x, 0

    old_r, r = a, b
    old_x, x = 1, 0

```

```

old_y, y = 0, 1

while r != 0:
    q = old_r // r
    old_r, r = r, old_r - q * r
    old_x, x = x, old_x - q * x
    old_y, y = y, old_y - q * y

g = abs(old_r)
# Normalize sign so g is positive
if old_r < 0:
    old_x, old_y = -old_x, -old_y

return g, old_x, old_y

gcd = extended_gcd_euclid(45, 10)
print(gcd)

```

(5, 1, -4)

4.4 расширенный бинарный алгоритм Евклида.

```

def extended_gcd_binary(a: int, b: int):
    """
    Extended binary GCD.
    Returns (g, x, y) such that:
        g = gcd(a, b)
    """

```

```

    a*x + b*y = g
    """
    if a == 0:
        return abs(b), 0, 1 if b > 0 else -1 if b < 0 else 0
    if b == 0:
        return abs(a), 1 if a > 0 else -1 if a < 0 else 0, 0

    # Work with positive values for the binary part, but keep originals for identity.
    a0, b0 = a, b
    a, b = abs(a), abs(b)

    # Remove common factors of 2
    shift = 0
    while ((a | b) & 1) == 0:
        a >>= 1
        b >>= 1
        shift += 1

    # u = a, v = b
    u, v = a, b
    A, B = 1, 0
    C, D = 0, 1

    while u != 0:
        # while u even
        while (u & 1) == 0:
            u >>= 1
            if (A & 1) == 0 and (B & 1) == 0:

```

```

        A >>= 1
        B >>= 1
    else:
        A = (A + b) >> 1
        B = (B - a) >> 1

# while v even
while (v & 1) == 0:
    v >>= 1
    if (C & 1) == 0 and (D & 1) == 0:
        C >>= 1
        D >>= 1
    else:
        C = (C + b) >> 1
        D = (D - a) >> 1

# subtract
if u >= v:
    u = u - v
    A = A - C
    B = B - D
else:
    v = v - u
    C = C - A
    D = D - B

g = v << shift # restore factors of 2

```

```
# Now we have:  $a \cdot C + b \cdot D = \gcd(a, b)$  where  $a, b$  are abs() reduced ones.  
# Fix signs according to original  $a_0, b_0$ :  
x, y = C, D  
if  $a_0 < 0$ :  
    x = -x  
if  $b_0 < 0$ :  
    y = -y  
  
return g, x, y
```

5 Выводы

Изученил и разработал методы программной реализации Алгоритмов вычисления наибольшего общего делителя.

Список литературы

https://en.wikipedia.org/wiki/Euclidean_algorithm